Sistemas de Informação



Disciplina: Programação I

Prof. Renê Rodrigues Veloso

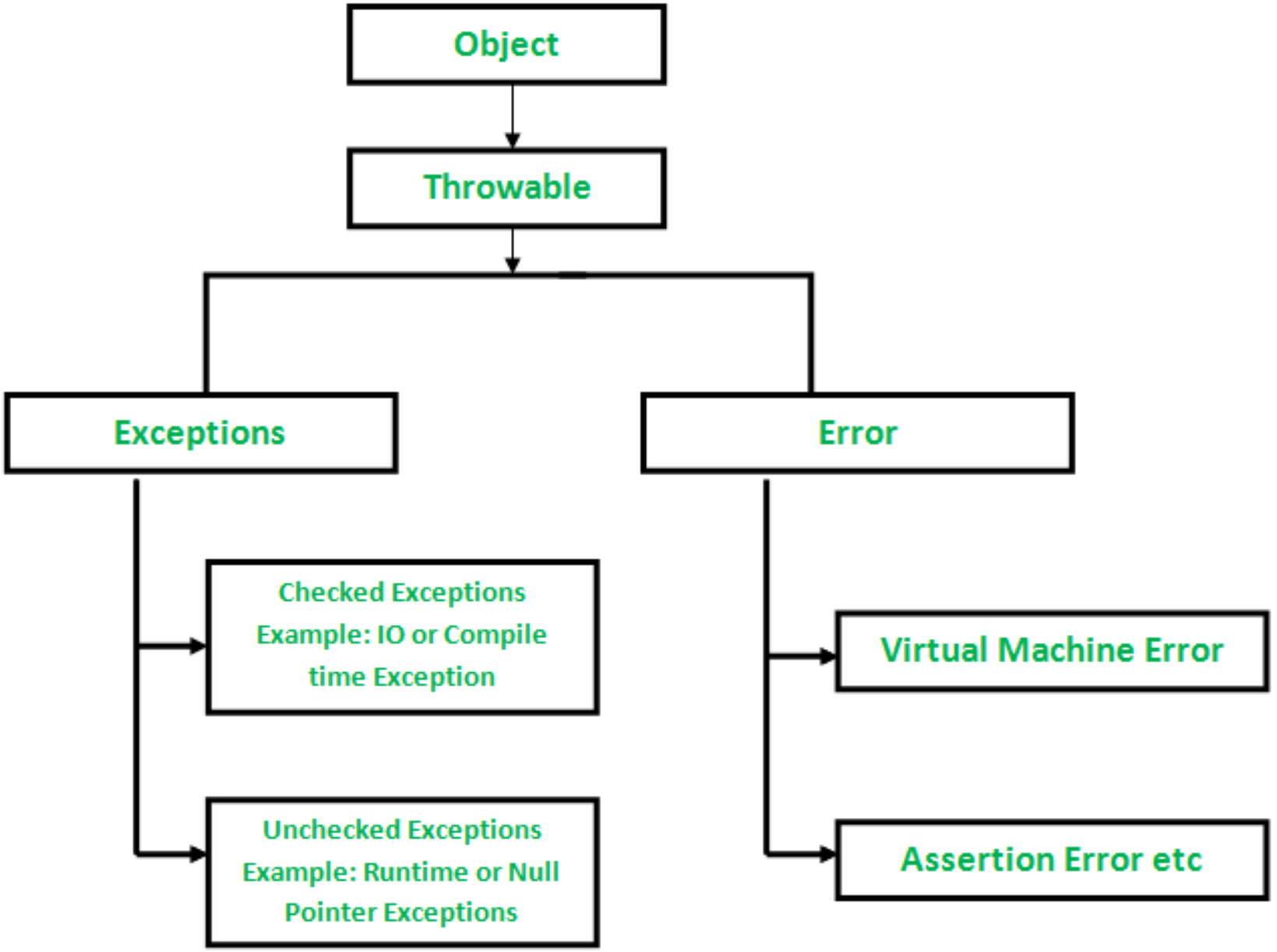
Exceções

Exceções são eventos inesperados que ocorrem durante a execução de um programa.

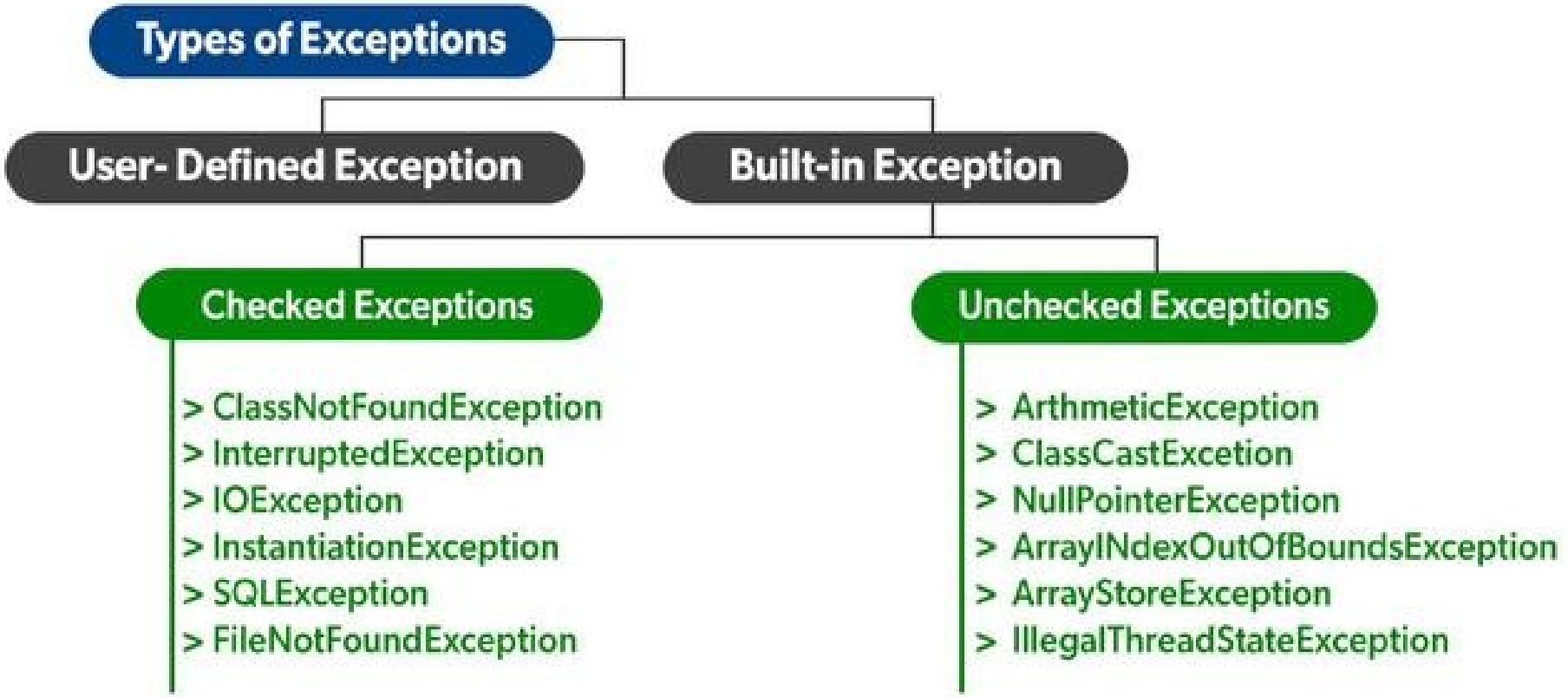
Pode ser o resultado de uma condição de erro ou de uma simples entrada inesperada.

Podem ser lançadas (ou disparadas) pelo ambiente de execução ou de forma explícita no código.

Exceções


Object
Throwable
Exceptions
Error
Checked Exceptions
Example: IO or Compile time Exception
Unchecked Exceptions
Example: Runtime or Null Pointer Exceptions
Virtual Machine Error
Assertion Error etc

Exceções


Types of Exceptions
User- Defined Exception
Built-in Exception
Checked Exceptions
> ClassNotFoundException
> InterruptedException
> IOException
> InstantiationException
> SQLException
> FileNotFoundException
Unchecked Exceptions
> ArthmeticException
> ClassCastExcetion
> NullPointerException
> ArrayINdexOutOfBoundsException
> ArrayStoreException
> IllegalThreadStateException

Exceções

- Built-in exceptions são exceções que estão disponíveis nas bibliotecas. Essas exceções são adequadas para explicar certas situações de erros.

- Checked Exceptions: checadas em tempo de compilação.
- Unchecked Exceptions: em tempo de execução.

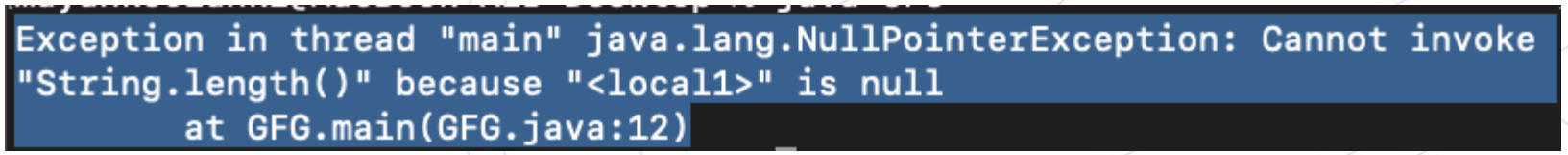
https://www.oracle.com/br/technical-resources/article/java/erros-java-exceptions.html
https://www.geeksforgeeks.org/checked-vs-unchecked-exceptions-in-java/

Lançando exceções

```
throw new tipo_da_exceção(parâmetros)
```

```
public static void imprimeString()
{
        String str = null;

        System.out.println(str.length());
}
```



Lançando exceções

`throw new` *`tipo_da_exceção(parâmetros)`*

```
public static void imprimeString()
{
        String str = null;

        if(str == null)
            throw new NullPointerException("string nula");

        System.out.println(str.length());
}
```

```
Exception in thread "main"
java.lang.NullPointerException: string nula
    at Main.imprimeString(Main.java:5)
    at Main.main(Main.java:15)
exit status 1
```

Lançando exceções

A cláusula *throws*

Especifica as exceções que podem ser lançadas por um método.

```
public static void imprimeString() throws NullPointerException
{
	String str = null;

	System.out.println(str.length());
}
```

Lançando exceções

A cláusula *throws*

Especifica as exceções que podem ser lançadas por um método.

```
public tipo método() throws Exceção1, Exceçao2, ...
{
    //...
}
```

Regra geral

Métodos não devem tratar as próprias exceções, mas apenas repassar para quem os chamou.

Capturando Exceções

Quando uma exceção é lançada, ela deve ser capturada ou o programa se encerrará.

A forma geral de se lidar com exceções é **“tentar”** executar um trecho de código que tenha a possibilidade de lançar uma exceção.

Se a exceção for lançada, então ela será **capturada**, fazendo com que o fluxo de controle desvie para um bloco **catch** que contém o código que trata a exceção.

Sintaxe try-catch

```
try
   bloco_a_ser_tentado…
catch(tipo_da_exceção_1 variável_1)
   bloco_de_tratamento_1
catch(tipo_da_exceção_2 variável_2)
   bloco_de_tratamento_2
...
finally
   bloco_sempre_executado
```

Sintaxe try-catch

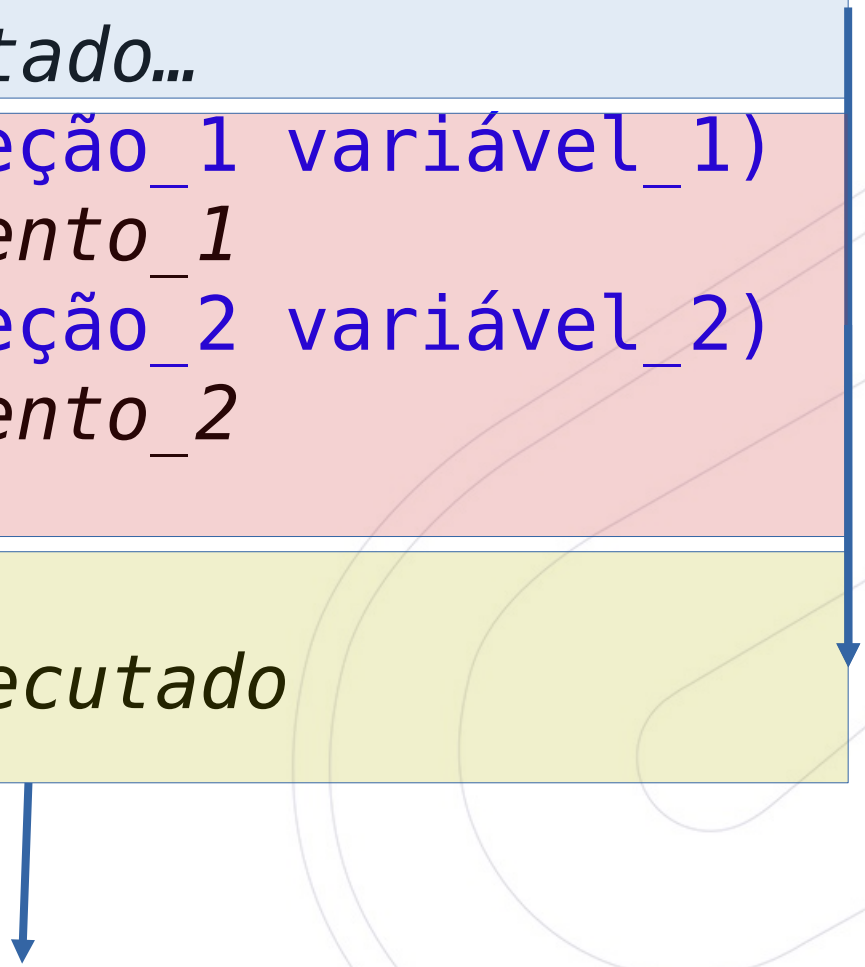
```
try
   bloco_a_ser_tentado…
catch(tipo_da_exceção_1 variável_1)
   bloco_de_tratamento_1
catch(tipo_da_exceção_2 variável_2)
   bloco_de_tratamento_2
...
finally
   bloco_sempre_executado
```

deve haver ao menos um

opcional

Sintaxe try-catch (fluxo de execução)

```
try
  bloco_a_ser_tentado…
catch(tipo_da_exceção_1 variável_1)
  bloco_de_tratamento_1
catch(tipo_da_exceção_2 variável_2)
  bloco_de_tratamento_2
...
finally
  bloco_sempre_executado
```



Exemplo

```
public static int divide(int a,int b){
    try {
        System.out.println("tentando...");
        int c = a/b;
        return c;
    }
    catch(ArithmeticException e){
      System.out.println("Erro: "+e);
    }
    finally{
      System.out.println("no finally.");
    }
    return 0;
  }
```

Criando novas exceções

Podemos estender as classes de exceções e criar exceções personalizadas:

```
package exemplos_aula06;

public class IndiceInvalidoException extends Exception {
  public IndiceInvalidoException(String msg) {
    super(msg);
  }
}
```

Criando novas exceções

Podemos estender as classes de exceções e criar exceções personalizadas:

```
package exemplos_aula06;

public interface List<T>{
  ...
  void insert(T valor, int pos) throws IndiceInvalidoException;
  ...
}
```

Criando novas exceções

```
public class ListaEncadeadaSimples<T> implements List<T>{
...

  public void insert(T valor, int pos) throws IndiceInvalidoException {
      ...
      else if(pos > tamanho || pos < 0)
          throw new IndiceInvalidoException("Indice invalido:"+pos);
      ...
   }
...
}
```

Criando novas exceções

```
class Main {
  public static void main(String[] args) {
    List<Integer> l = new ListaEncadeadaSimples<>();
    l.append(1);
    l.append(2);
    l.append(3);
    try{
        l.insert(4, -1);
    }
    catch(IndiceInvalidoException e){
        System.out.println(e);
    }
  }
}
```